

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Nik Katanović Leban

**Spletna aplikacija za prodajo in
evalvacijo avtomobilov**

DIPLOMSKO DELO

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE

MENTOR: viš. pred. dr. Igor Rožanc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Nik Katanović Leban

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Vrsta naloge: Diplomski naloga na visokošolskem programu prve stopnje
Računalništvo in informatika

Mentor: viš. pred. dr. Igor Rožanc

Opis:

Besedilo teme diplomskega dela študent prepíše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Web application for car sales and evaluation

Description:

The student shall transcribe the text of the diploma thesis topic from the student information system, where it was entered by the mentor. In a few sentences, they will describe what is expected of the candidate's thesis. What the objectives are, what methods should be used, and perhaps they will also list the key literature.

Rad bi se zahvalil svojemu mentorju Igorju Rožancu, za vso aktivno pomoč in svetovanje pri razvoju te diplomske naloge. Zahvalil bi se tudi vsem sodelavcem iz podjetja CargoX, ki so z mano delili svoje znanje in izkušnje, ter mi omogočili, da sem se lahko lotil tako zahtevnega projekta. Posebna zahvala gre moji družini, ki me je ves čas podpirala in mi stala ob strani.

Kazalo

Povzetek

Abstract

1	Uvod	1
1.1	Opis arhitekture sistema	2
2	Pregled področja in sorodnih del	5
3	Tematsko ozadje	7
3.1	Spletne aplikacije	7
3.2	Relacijske podatkovne baze	8
3.3	Mikrostoritvena arhitektura	9
3.4	Zajem podatkov iz spletnih virov	10
4	Metodologija	13
4.1	Načrtovanje podatkovne baze	14
4.2	Arhitektura zalednega sistema	17
4.3	Arhitektura uporabniškega vmesnika	21
4.4	Evalvacijski sistem	25
4.5	Infrastruktura in avtomatizacija	25
5	Vključevanje programske kode in algoritmov	27
5.1	Vključevanje algoritmov (psevdokode)	28

6 Kaj pa literatura?	31
7 Sklepne ugotovitve	33
Literatura	35
Članki v revijah	35
Članki v zbornikih konferenc	37
Ostala literatura	38
Dodatek	43

Povzetek

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Avtor: Nik Katanović Leban

Diplomsko delo obravnava problem določanja primerne tržne vrednosti rabljenega avtomobila. Pri prodaji ali nakupu vozila uporabniki pogosto nimajo zadostnega pregleda nad trenutnim stanjem na trgu, zato težko ocenijo, ali je določena cena primerna. Posledično lahko vozilo kupijo po previsoki ceni ali ga prodajo pod njegovo dejansko tržno vrednostjo. Za reševanje tega problema sem razvil spletno aplikacijo za objavo in pregled avtomobilskih oglasov, ki uporabnikom omogoča enostavno in varno prodajo ter nakup vozil. Poleg tega je bil implementiran sistem za avtomatsko ocenjevanje vrednosti vozil. Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in primerjavi vozila s podobnimi vozili na trgu. Pri oceni vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik in prevoženi kilometri. Za pridobivanje podatkov so bili uporabljeni javno dostopni podatkovni viri ter postopki samodejnega zajema podatkov s spletnih strani za prodajo vozil. Zbrane podatke sistem obdela in uporabi za izračun ocenjene tržne vrednosti vozila. Rezultat diplomskega dela je delujoča spletna aplikacija s podporo za avtomatsko vrednotenje vozil, ki uporabnikom pomaga pri sprejemanju bolj informiranih odločitev pri nakupu ali prodaji avtomobila.

Ključne besede: spletna aplikacija, avtomobili, vrednotenje, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Abstract

Title: Web application for car sales and evaluation

Author: Nik Katanović Leban

This work addresses the problem of determining the appropriate market value of a used vehicle. When buying or selling a car, users often lack sufficient insight into current market conditions, making it difficult to assess whether a listed price is fair. As a result, vehicles may be purchased at inflated prices or sold below their actual market value. To address this problem, a web application for publishing and browsing vehicle listings was developed, enabling users to buy and sell vehicles in a simple and secure manner. In addition, an automated vehicle valuation system was implemented. The system is based on collecting data from external sources, normalizing the acquired data, and comparing a vehicle with similar vehicles currently available on the market. The valuation process takes into account key vehicle characteristics, including brand, model, year of first registration, and mileage. The required data was obtained from publicly available datasets and through automated data collection from vehicle marketplace websites. The collected data is processed and used to estimate the market value of a vehicle. The result of this thesis is a functional web application with integrated vehicle valuation support that helps users make more informed decisions when buying or selling used vehicles.

Keywords: web application, automobiles, evaluation, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Poglavje 1

Uvod

Nakup in prodaja rabljenih avtomobilov predstavljata pomemben del avtomobilskega trga. Pri tem se uporabniki pogosto srečujejo s težavo določanja primerne tržne vrednosti vozila, saj nimajo zadostnega pregleda nad trenutno ponudbo in gibanjem cen na trgu. Najpogostejši pristop je primerjava z obstoječimi oglasi podobnih vozil, vendar takšna metoda ni vedno zanesljiva. Oglasi lahko vsebujejo nerealno visoke ali nizke cene, uporabniki pa pogosto nimajo dovolj znanja in izkušenj, da bi lahko ocenili dejansko vrednost vozila. Posledično lahko vozilo prodajo pod njegovo tržno vrednostjo ali ga kupijo po previsoki ceni.

Na trgu že obstajajo rešitve za ocenjevanje vrednosti vozil, vendar so mnoge plačljive, prilagojene tujim trgom ali pa temeljijo na zastarelih podatkih. Zaradi tega njihove ocene pogosto ne odražajo dejanskega stanja na slovenskem trgu rabljenih vozil.

Cilj diplomskega dela je razvoj spletne aplikacije za objavo in pregled avtomobilskih oglasov ter razvoj sistema za samodejno ocenjevanje vrednosti vozil. Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in analizi primerljivih vozil na trgu. Pri določanju ocenjene vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik prve regi-

stracije in število prevoženih kilometrov. Dolgoročni cilj sistema je privabiti zadostno število uporabnikov, ki na platformi objavljajo lastne avtomobilske oglase, tako da bo potreba po podatkih iz zunanjih virov postopno upadla, tržna vrednost vozil pa bo mogoče ocenjevati na podlagi oglasov zbranih znotraj platforme same.

Hipoteza diplomskega dela je, da je mogoče s pomočjo zbranih tržnih podatkov in primerjalne analize podobnih vozil uporabniku ponuditi uporabno in dovolj natančno oceno tržne vrednosti vozila. Dodatni cilj je preveriti, ali integracija takšnega sistema v spletno aplikacijo izboljša uporabniško izkušnjo pri nakupu in prodaji vozil.

V poglavju ?? je predstavljena analiza problema, pregled obstoječih rešitev in uporabljene tehnologije. Poglavje ?? opisuje načrtovanje sistema in arhitekturo aplikacije. Implementacija posameznih komponent, vključno s sistemom za vrednotenje vozil, je predstavljena v poglavju ?. V poglavju ?? so opisani postopki testiranja in rezultati evalvacije sistema. Diplomsko delo se zaključuje s sklepom in predlogi za nadaljnje izboljšave.

1.1 Opis arhitekture sistema

Spletna aplikacija je sestavljena iz uporabniškega vmesnika (odjemalec), strežniškega dela aplikacije (zaledni sistem) in ločenega sistema za vrednotenje vozil. Uporabniški vmesnik je razvit z uporabo knjižnice React in programskega jezika TypeScript v okolju Node.js različice 24. Zaledni sistem je razvit z uporabo ogrodja Express v programskem jeziku JavaScript v okolju Node.js različice 24. Sistem za vrednotenje vozil je implementiran kot ločena storitev v programskem jeziku Python različice 3.11. Za izvajanje posameznih komponent je uporabljena platforma Docker.

Docker kontejnerji:

- Zaledni sistem
- Postgres:16
- redis:7
- mailhog
- Evalvacijski sistem

Razvoj aplikacije je potekal v razvojnem okolju Visual Studio Code. Za avtomatizacijo preverjanja kode, izvajanja testov ter gradnje vsebnikov Docker je bil uporabljen sistem GitHub Actions, medtem ko so se periodična opravila izvajala s pomočjo časovno načrtovanih opravil (cron). Za gostovanje aplikacije je bil uporabljen virtualni strežnik ponudnika Hetzner. Namestitev in upravljanje aplikacije na strežniku je bilo izvedeno s pomočjo platforme Dokploy.

Poglavje 2

Pregled področja in sorodnih del

Na področju prodaje rabljenih vozil obstaja več spletnih aplikacij, ki uporabnikom omogočajo objavo in pregled avtomobilskih oglasov. Med najbolj znanimi evropskimi ponudniki sta mobile.de in AutoScout24, ki združujeta veliko število oglasov iz različnih držav ter uporabnikom omogočata filtriranje vozil po številnih kriterijih. Za slovenski trg je najbolj razširjena platforma Avto.net, ki predstavlja osrednje mesto za objavo in iskanje rabljenih vozil. Nekatere izmed omenjenih platform uporabnikom ponujajo tudi osnovno oceno primernosti cene vozila. Mobile.de na primer pri posameznih oglasih prikazuje informacijo o tem, ali je cena vozila glede na primerljive oglase ugodna, primerna ali previsoka. Takšen pristop uporabnikom omogoča hitrejšo orientacijo na trgu, vendar praviloma ne ponuja možnosti samostojnega vrednotenja vozila na podlagi uporabniško vnesenih podatkov.

Poleg spletnih tržnic obstajajo tudi specializirane storitve za vrednotenje vozil. Med najbolj znanimi je Eurotax, ki se pogosto uporablja v avtomobilski industriji, zavarovalništvu in pri trgovcih z vozili. Njegove ocene temeljijo na obsežnih zbirkah podatkov o vozilih in tržnih cenah, vendar je dostop do storitve praviloma plačljiv. Zaradi poslovnega modela in omejenega dostopa podrobna analiza delovanja sistema ni mogoča. Na ameriškem trgu sta razširjeni storitvi Kelley Blue Book in CARFAX. Kelley Blue Book uporabnikom omogoča oceno vrednosti vozila na podlagi tehničnih podatkov in

tržnih analiz, medtem ko CARFAX poleg vrednotenja ponuja tudi vpogled v zgodovino vozila. Obe storitvi sta prilagojeni predvsem severnoameriškemu trgu in zato nista neposredno uporabni za slovenski trg rabljenih vozil.

Pregled obstoječih rešitev je pokazal, da večina platform bodisi omogoča prodajo vozil brez naprednega sistema za vrednotenje bodisi ponuja ločene plačljive storitve za ocenjevanje vrednosti vozil. Namen razvite rešitve je združiti obe funkcionalnosti v enotni spletni aplikaciji, ki uporabnikom omogoča objavo vozil, pregled trga in samodejno oceno vrednosti vozila na podlagi primerljivih vozil, dostopnih na trgu.

Poglavje 3

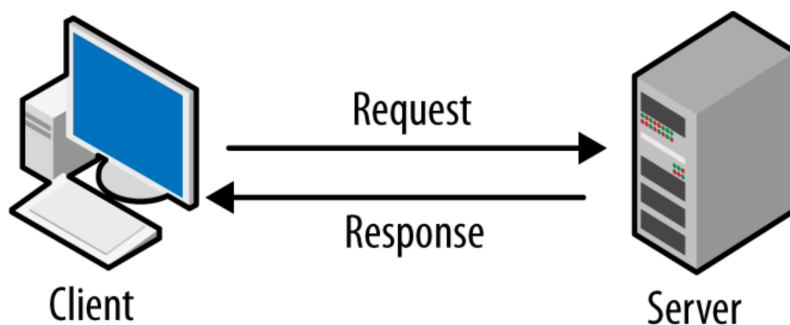
Tematsko ozadje

3.1 Spletne aplikacije

Spletne aplikacije sestavljata uporabniški vmesnik (odjemalec) in strežniški del aplikacije (zaledni sistem). Takšna arhitektura omogoča ločitev skrbi, kjer vsak del sistema prevzame svojo vlogo v celotni aplikaciji [35]. Uporabniški vmesnik omogoča prikaz podatkov ter interakcijo uporabnika z aplikacijo. Njegova glavna naloga je zagotavljanje preglednega, odzivnega in uporabniku prijaznega dostopa do funkcionalnosti sistema. Poleg prikaza podatkov skrbi tudi za zajem uporabniških vnosov, komunikacijo s strežniškim delom aplikacije ter lokalno shranjevanje določenih podatkov v spletnem brskalniku, kar prispeva k boljši odzivnosti in uporabniški izkušnji [15]. Sodobni uporabniški vmesniki so pogosto zasnovani na komponentni arhitekturi. Komponenta predstavlja samostojen in ponovno uporaben gradnik uporabniškega vmesnika, ki ga je mogoče vključiti na različnih mestih v aplikaciji. Takšen pristop izboljšuje modularnost sistema, poenostavlja vzdrževanje kode ter omogoča boljšo ponovno uporabo funkcionalnosti [27].

Strežniški del aplikacije je odgovoren za obdelavo zahtevkov, ki jih prejme od odjemalca, ter za vračanje ustreznih odgovorov. Poleg tega izvaja poslovno logiko sistema, upravlja z dostopnimi pravicami uporabnikov ter skrbi za avtentikacijo in avtorizacijo. Strežniški del je prav tako odgovoren za ko-

munikacijo s podatkovno bazo, kjer se hranijo trajni podatki aplikacije [14]. V sodobnih spletnih aplikacijah strežniški del pogosto omogoča dostop do podatkov preko programskih vmesnikov (vmesnik API). Eden izmed najpogostejše uporabljenih arhitekturnih pristopov je REST, ki temelji na brezstanovnem (angl. *stateless*) načinu komunikacije med sistemi in uporablja standardne HTTP metode za izvajanje operacij nad viri [14]. Poleg osnovne funkcionalnosti lahko strežniški del vključuje tudi dodatne mehanizme, kot so omejevanje števila zahtevkov (angl. *rate limiting*), predpomnjenje podatkov ter beleženje dogodkov, kar prispeva k večji varnosti, stabilnosti in zmogljivosti sistema [13].



Slika 3.1: Arhitektura spletne aplikacije tipa odjemalec–strežnik [8].

3.2 Relacijske podatkovne baze

Relacijske podatkovne baze predstavljajo enega izmed najpogostejše uporabljenih pristopov za shranjevanje podatkov v spletnih aplikacijah. Temeljijo na relacijskem modelu, pri katerem so podatki organizirani v tabele, ki predstavljajo posamezne entitete. Vsaka entiteta vsebuje attribute, s katerimi opisujemo njene lastnosti, posamezne vrstice pa predstavljajo konkretne primerke entitete. Relacijski model omogoča strukturirano in konsistentno shranjevanje podatkov ter učinkovito izvajanje poizvedb nad velikimi količinami podatkov [1]. Povezave med entitetami vzpostavimo s primarnimi in tujimi ključi. Primarni ključ enolično identificira posamezen zapis v tabeli, medtem

ko tuji ključ omogoča povezovanje podatkov med različnimi tabelami. Na ta način zmanjšamo podvajanje podatkov ter zagotovimo referenčno integriteto podatkovne baze [1].

Pri razvoju sistema smo izbrali PostgreSQL, ki predstavlja odprtokodni objektno-relacijski sistem za upravljanje podatkovnih baz. PostgreSQL temelji na standardu SQL in ga razširja z naprednimi funkcionalnostmi, ki omogočajo učinkovito delo tako s strukturiranimi kot tudi s polstrukturiranimi podatki [39]. Med pomembnejšimi lastnostmi sistema PostgreSQL so podpora za transakcije ACID, napredne poizvedbe SQL, replikacija podatkov, mehanizmi za zagotavljanje integritete podatkov ter podpora za podatkovni format JSON oziroma JSONB. Zaradi teh funkcionalnosti je PostgreSQL primeren za razvoj sodobnih spletnih aplikacij, ki zahtevajo visoko zmogljivost, zanesljivost in možnost nadaljnje razširljivosti sistema [39, 38].

3.3 Mikrostoritvena arhitektura

Mikrostoritvena arhitektura je arhitekturni vzorec, pri katerem se programski sistem razdeli na množico manjših, med seboj neodvisnih storitev, namesto da bi bil zgrajen kot ena tesno povezana enota, znana kot monolitna arhitektura [10]. Vsaka mikrostoritev implementira natančno določeno poslovno funkcionalnost in je razvita, nameščena ter skalirana neodvisno od preostalih storitev v sistemu. Komunikacija med posameznimi mikrostoritvami praviloma poteka prek dobro definiranih vmesnikov, kot so vmesniki API, sporočilne vrste ali drugi mehanizmi za izmenjavo podatkov [30].

Razdelitev sistema na več manjših, neodvisnih komponent prinaša več prednosti v primerjavi z monolitno arhitekturo [30]. Posamezne mikrostoritve je lažje testirati, saj imajo omejen in jasno določen obseg delovanja. Prav tako je olajšano odkrivanje in odpravljanje napak, saj je napako mogoče hitreje lokalizirati znotraj posamezne storitve, namesto da bi jo iskali znotraj celotnega sistema. Neodvisnost storitev dodatno omogoča lažje vzdrževanje, saj je mogoče posodobiti ali zamenjati eno mikrostoritev, ne da bi to vplivalo

na delovanje ostalih mikrostoritev v sistemu, pod pogojem, da vmesnik za komunikacijo med njimi ostane nespremenjen [10].

3.4 Zajem podatkov iz spletnih virov

Zajem podatkov iz spletnih virov predstavlja temeljni korak pri vrednotenju rabljenih vozil na trgu. Kakovost, obseg in svežost zbranih podatkov neposredno vplivajo na zanesljivost modelov za ocenjevanje cen. [24]. V splošnem ločimo tri pristope za pridobivanje podatkov iz spleta: izkoriščanje javno dostopnih podatkovnih zbirk, samodejni zajem podatkov s spletnih strani (ang. *web scraping*) ter dostop prek vmesnikov API [24].

3.4.1 Javno dostopne podatkovne zbirke

Javno dostopne podatkovne zbirke predstavljajo enega najprimernejših virov za zbiranje strukturiranih podatkov, saj so podatki v njih že organizirani, kategorizirani in pripravljeni za neposredno analitično obdelavo [20]. Primeri takšnih zbirk so odprti vladni portali, kot sta Statistični urad Republike Slovenije (SURS) [37] in Eurostat [12], akademski repozitoriji ter portali prometnih agencij. Slabost tega pristopa je omejena razpoložljivost aktualnih tržnih podatkov. Javne zbirke navadno ne vsebujejo oglasov z aktualnimi prodajnimi cenami vozil, ki so nujno potrebni za ocenjevanje tržne vrednosti [17]. Javne zbirke navadno ne vsebujejo oglasov z aktualnimi prodajnimi cenami vozil, ki so nujno potrebni za ocenjevanje tržne vrednosti [17]. Za tuje trge sicer obstajajo javno dostopne zbirke oglasov, pridobljenih z oglašnih portalov, npr. nemški trg (eBay Kleinanzeigen) [32], vendar za slovenski trg takšna javna zbirka ne obstaja.

3.4.2 Samodejni zajem podatkov s spletnih strani

Samodejni zajem podatkov s spletnih strani je postopek avtomatiziranega pridobivanja strukturiranih informacij iz spletnih strani z razčlenjevanjem

njihove vsebine [24]. Ta pristop se pogosto uporablja pri zbiranju oglasov za prodajo rabljenih vozil, saj je na voljo veliko portalov, ki vsebujejo obsežne zbirke takšnih oglasov. Iz vsakega oglasa je mogoče pridobiti strukturirane attribute vozila, kot so znamka, model, letnik izdelave, prevoženi kilometri, vrsta goriva, tip menjalnika in oglašena prodajna cena [17]. Zajem poteka z orodji za avtomatizirano brskanje po spletnih straneh, ki simulirajo obisk uporabnika, ter z razčlenjevanjem strukture dokumenta HTML za izluščitev relevantnih podatkovnih polj [24].

3.4.3 Razčlenjevanje vsebine HTML

Spletna stran je v osnovi dokument v jeziku HTML (ang. *HyperText Markup Language*), ki je organiziran v drevesno strukturo DOM (ang. *Document Object Model*) [31]. Vsak element dokumenta, kot so naslovi, odstavki in podatkovne celice, je v tej strukturi predstavljen kot vozlišče, ki ima lahko starševska in podrejena vozlišča. Orodja za razčlenjevanje (ang. *parsers*), kot sta knjižnici BeautifulSoup in lxml v jeziku Python, to strukturo pretvorijo v obliko, primerno za programsko obdelavo.

Za izluščevanje podatkovnih elementov iz razčlenjene strukture se v praksi najpogosteje uporabljata dve vrsti izbirnikov: izbirniki CSS in izbirniki XPath [7]. Izbirniki CSS so bili prvotno zasnovani za oblikovanje spletnih strani, a jih pri zajemu podatkov preusmerimo v poizvedovanje po strukturi DOM [31]. Izbirniki XPath (ang. *XML Path Language*) pa so namensko poizvedovalni jezik za navigacijo po drevesnih strukturah XML in HTML, ki poleg preprostega iskanja po oznakah in razredih podpira tudi premikanje navzgor po drevesu, iskanje po vsebini besedilnih vozlišč ter vgrajene funkcije, kot sta `contains()` in `position()` [7]. V kontekstu zajemanja podatkov iz avtomobilskih oglasnih portalov to pomeni, da z izbirniki iz razčlenjenega dokumenta izluščimo posamezna podatkovna polja oglasa, kot so naziv vozila, cena, letnik, prevoženi kilometri in vrsta goriva.

Poglavje 4

Metodologija

Cilj diplomske naloge je bila izdelava spletne aplikacije, v kateri bodo uporabniki lahko objavljali oglase svojih avtomobilov. Pred začetkom razvoja smo morali izbrati ustrezne tehnologije, kar smo storili na podlagi lastnih izkušenj in predznanja ter pregleda različnih forumov in strokovnih člankov. Ko smo zaključili razvoj spletne aplikacije, smo se lotili razvoja sistema za vrednotenje avtomobilov, napisanega v programskem jeziku Python. Python smo izbrali, ker je zelo primeren za obdelavo podatkov in razpolaga z bogatim ekosistemom knjižnic, namenjenih analizi podatkov.

Pri izbiri orodij za razvoj uporabniškega vmesnika smo se odločili za ogrodje React. Odločitev je deloma temeljila na želji po pridobitvi novih znanj v okviru diplomskega dela, saj z Reactom pred tem nismo imeli izkušenj. React je eno izmed najpopularnejših ogrodij za razvoj uporabniških vmesnikov z eno največjih skupnosti, kar zagotavlja kakovostno dokumentacijo in dolgoročno podporo. Skupaj z Reactom smo uporabili jezik TypeScript, ki je nadgradnja jezika JavaScript [28]. TypeScript razvijalce prisili k eksplicitni deklaraciji tipov spremenljivk, kar bistveno zmanjša možnost napak med razvojem.

Za razvoj zalednega sistema smo izbrali ogrodje Express.js, zgrajeno na platformi Node.js, ki omogoča hitro in enostavno izdelavo spletnih aplikacij. Odločitev je temeljila na predhodnih izkušnjah z ogrodjem ter na njegovi eno-

stavni integraciji z vmesniki REST API. Zaledni sistem smo razvili v jeziku JavaScript, kar se je naknadno izkazalo za manj optimalno odločitev. Jezik TypeScript bi z dodatno varnostjo tipov prispeval k boljši zanesljivosti in vzdržljivosti kode.

4.1 Načrtovanje podatkovne baze

4.1.1 Načrtovanje podatkovne baze aplikacije

Pri načrtovanju podatkovne baze smo morali natančno premisliti, kako shranjujemo oglase avtomobilov, saj je to ključnega pomena za učinkovito delovanje odločitvenega modela. Previsoka stopnja normalizacije bi upočasnila poizvedbe, prenizka pa bi vodila do podvajanja podatkov in posledičnih neskladnosti v bazi [11].

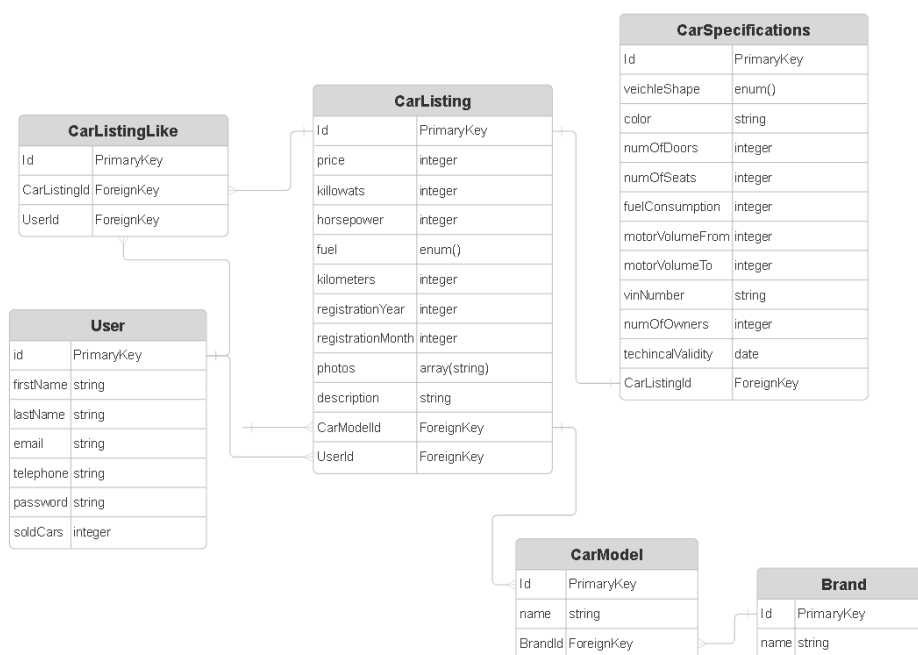
Entiteti `Brand` in `CarModel` sta izločeni v ločeni tabeli, ker med njima obstaja hierarhično razmerje ena-proti-mnogo: ena znamka (`Brand`) ima lahko poljubno število modelov (`CarModel`), vsak model pa pripada natanko eni znamki. Shranjevanje znamk in modelov kot vrednosti tipa `enum` ali prostega niza bi bilo neprimerno iz dveh razlogov. Prvič, število možnih vrednosti je preveliko za statično definicijo z `enum`. Drugič, prosti niz bi omogočal podvojene ali nekonsistentne vnose – na primer »Audi A3« in »audi a3 Sportback« bi bila obravnavana kot različni entiteti, čeprav gre za isti model. Z relacijsko vezavo prek tujih ključev je zagotovljen referenčni nadzor nad vnesenimi vrednostmi ter enotnost podatkov.

Entiteta `CarSpecifications` je namerno ločena od entitete `CarListing`, ker vsebuje tehnične podrobnosti vozila, ki niso relevantne pri vsakem poizvedovanju. Pri prikazu seznama oglasov (npr. v podatkovni tabeli) zadostujejo le osnovni podatki iz entitete `CarListing` (cena, prevoženi kilometri, letnik ...), specifikacije pa se naložijo le ob ogledu posameznega oglasa. Ta ločitev zmanjša količino podatkov, ki se prenesejo iz podatkovne baze pri pogostih

poizvedbah, in s tem izboljša odzivnost aplikacije.

Entiteta `CarListingLike` predstavlja asociativno tabelo, ki realizira razmerje mnogo-proti-mnogo med uporabniki in oglasi: en uporabnik lahko označi kot všečkano poljubno število oglasov, en oglas pa je lahko všečkan pri poljubnem številu uporabnikov. Tabela vsebuje tuji ključ na entiteto `User` in tuji ključ na entiteto `CarListing`, kar zagotavlja referenčno integriteto in preprečuje podvojene vnose.

Entiteta `User` hrani standardne podatke o registriranih uporabnikih sistema.



Slika 4.1: ER-diagram glavnih entitet aplikacije.

4.1.2 Načrtovanje podatkovne baze odločitvenega modela

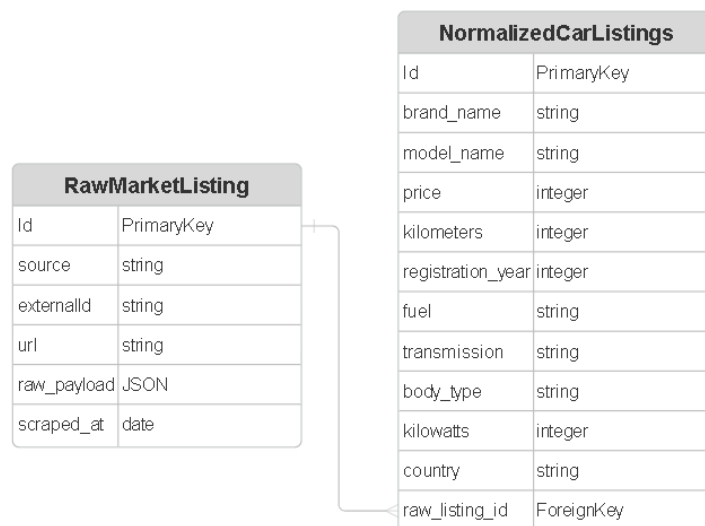
V okviru iste podatkovne baze smo dodali ločeno shemo **market**, ki vsebuje dve tabeli, namenjeni zbiranju in obdelavi tržnih podatkov za sistem vredno-

tenja avtomobilov.

Tabela `RawMarketListing` služi kot surov arhiv vseh oglasov, pridobljenih iz zunanjih virov – javno dostopnih zbirk podatkov, spletnih vmesnikov API in z zajemanjem spletnih strani. Tabela vsebuje naslednja polja: `source` identificira izvor podatkov (npr. ime portala ali vmesnika API), `externalId` je identifikator oglasa na izvornem sistemu, ki je določen statično v kodi in preprečuje podvojene vnose, `url` hrani neposredno povezavo do oglasa, `scraped_at` je časovni žig trenutka zajema podatkov, `raw_payload` pa vsebuje izvorni odgovor v obliki JSON brez kakršnekoli obdelave. Izvorni odgovor je namerno shranjen v celoti, kar omogoča kasnejšo ponovno obdelavo brez ponovnega zajema podatkov, če se normalizacijska logika spremeni.

Tabela `NormalizedCarListing` vsebuje obdelano in normalizirano obliko surovega oglasa, vezano na tabelo `RawMarketListing` prek tujega ključa `raw_listing_id`. Razmerje med tabelama je ena-proti-mnogo: vsak surovi oglas ima lahko več normaliziranih zapisov. Polja so poenotena in tipizirana – cena, prevoženi kilometri in letnik so shranjeni kot `integer`, znamka, model, vrsta goriva, tip menjalnika, oblika karoserije in država pa kot `string`. Takšna poenotena struktura omogoča neposredno primerjavo oglasov iz različnih virov pri izračunu tržne vrednosti vozila.

Ločitev na dve tabeli sledi vzorcu cevovoda surovih in normaliziranih podatkov (angl. *raw/normalized pipeline*) [22]: surovi podatki ostanejo nedotaknjeni kot revizijska sled in osnova za morebitno ponovno obdelavo, normalizirana oblika pa je tista, ki se uporablja v analitičnih poizvedbah.



Slika 4.2: ER-diagram entitet Evalvacijskega sistema.

4.2 Arhitektura zalednega sistema

Zaledni sistem smo oblikovali z modularno arhitekturo, ki omogoča jasno ločitev odgovornosti (separation of concerns), kar olajša branje, vzdrževanje in nadaljnjo razširitev kode [9]. Kot ogrodje za zaledni sistem smo izbrali Express.js, minimalistično ogrodje za Node.js, ki zagotavlja temeljne funkcionalnosti za usmerjanje zahtev in upravljanje vmesnika HTTP, hkrati pa razvojnikom omogoča svobodno izbiro specifičnih knjižnic glede na različne zahteve aplikacije [5].

4.2.1 Modul za poizvedbe v podatkovni bazi

Za implementacijo podatkovnih modelov in poizvedkov na podatkovno bazo smo uporabili knjižnico Sequelize. Sequelize je vmesnik za spreminjanje objektno-relacijskih preslikav (ORM – Object-Relational Mapping), ki omogoča preslikavo objektov iz programske kode v relacijske strukture podat-

kovne baze [6]. ORM pristop izboljšuje razvoj, saj omogoča programerjem, da delajo z objekti namesto neposrednih poizvedb SQL, kar zmanjša napake in poveča berljivost kode [6]. Prednosti ORM pristopa vključujejo tipsko varnost, avtomatično generiranje poizvedb za različne podatkovne baze ter intuitivnejše upravljanje relacij med entitetami.

Podatkovni modeli so v Sequelize definirani kot razredi, ki predstavljajo tabele v bazi podatkov [6]. Vsak model ima attribute, ki ustrezajo stolpcem v tabeli, ter metode za izvajanje temeljnih operacij s podatki (CRUD – Create, Read, Update, Delete). Ključno pri oblikovanju modelov je tudi postavitvev indeksov na poljih, ki se pogosto pojavljajo v pogojih poizvedb (npr. ID avtomobila, izvor oglasa ali časovni žigi). Dobro zasnovani indeksi so bistvenega pomena za zagotavljanje hitrih odgovorov poizvedb pri velikih količinah podatkov [16]. Primer definirane modelne strukture je prikazan v Kodi 7.

Izsek kode 1: Koda za vzpostavitev povezave z bazo podatkov.

```
1 import { Sequelize } from 'sequelize';
2
3 const sequelize = new Sequelize(process.env.DB_NAME, process.env.DB_USER,
4   ↪ process.env.DB_PASSWORD, {
5   host: process.env.DB_HOST,
6   port: process.env.DB_PORT,
7   dialect: 'postgres',
8   logging: false,
9 });
10 export default sequelize;
```

4.2.2 Avtentikacija in avtorizacija uporabnikov

Za avtentikacijo uporabnikov smo implementirali sistem, ki temelji na žetonih JWT (JSON Web Token), kar omogoča varen in enostaven prenos podatkov o uporabnikovi pristnosti [21]. Žeton JWT je strukturiran dokument, sestavljen iz treh delov: glave (header), ki vsebuje metapodatke o vrsti kodiranja;

podatkov (payload), ki nosijo informacije o uporabniku; in digitalnega podpisa, ki zagotavlja celovitost in pristnost žetona [21].

Postopek avtentikacije je razdeljen na dve fazi. V fazi registracije uporabnik vnese svoje geslo, ki ga pred shranitvijo v podatkovno bazo zgoščimo z algoritmom bcrypt [34]. Bcrypt je specializirana kriptografska funkcija, namenjena zgoščevanju gesel, ki vključuje samodejno upoštevanje (salting) in je odpornejša na napade [34]. V fazi prijave uporabnik pošlje svoje geslo in e-poštni naslov. Zaledni sistem pridobi shranjeno zgoščeno geslo iz baze in ga primerja z novim zgoščenim geslom s pomočjo bcrypt funkcije za primerjavo. Če se gesla ujemata, zaledni sistem generira JWT žeton z informacijami o uporabniku in ga pošlje nazaj na uporabniški vmesnik, kjer ga uporabnik shrani v brskalniku (največkrat v shrambo seje – localStorage).

Za zaščito poti API-ja smo implementirali middleware za preverjanje žetonov. Vmesnik je funkcija, ki se izvede pred vsakim zahtevkom in preveri, ali je uporabnik poslal veljaven JWT žeton v zahtevku. Dodatna varnostna plast je nastavljena z časovnimi omejitvami žetona, ki zagotavlja, da je žeton veljaven samo za določen čas. Po preteku roka veljavnosti se mora uporabnik ponovno avtentificirati in pridobiti nov žeton, kar zmanjšuje tveganje ob morebitnem razkritju stareših žetonov [18].

Izsek kode 2: Koda za prijavo uporabnika.

```
1 export const loginUserService = async ({ email, password }) => {  
2   try {  
3     if (!email || !password) throw new MissingUserDataError();  
4  
5     const user = await User.findOne({ where: { email } });  
6  
7     if (!user) throw new AppError('Invalid email or password', 401);  
8  
9     const isPasswordValid = await bcrypt.compare(password, user.password);  
10  
11    if (!isPasswordValid) throw new AppError('Invalid email or password',  
    ↪ 401);
```

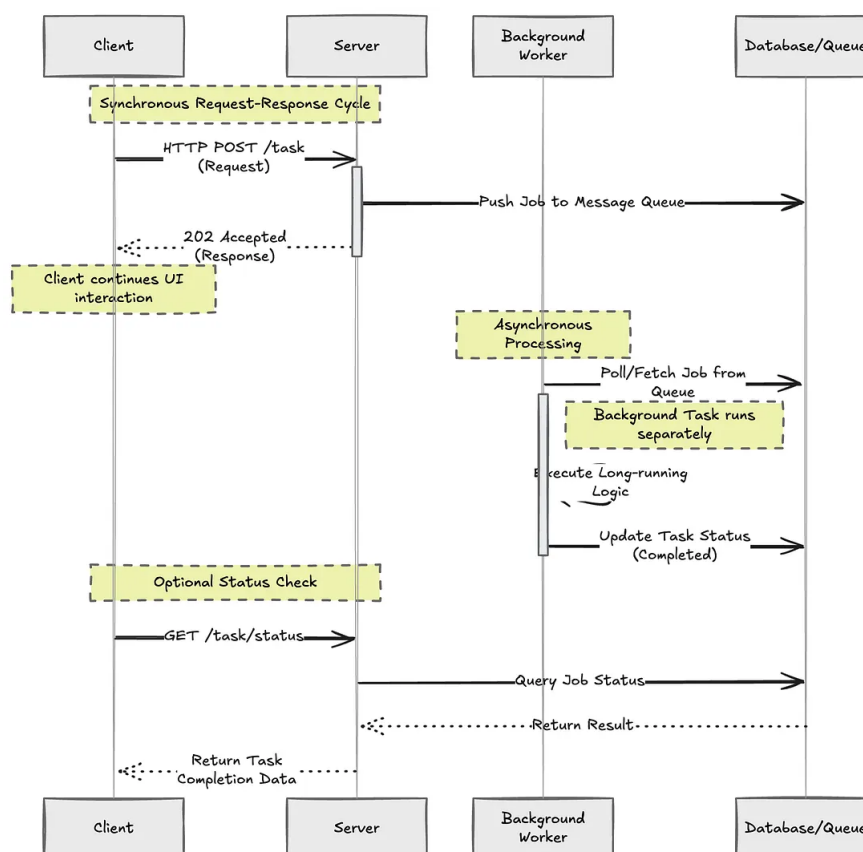
```
12
13     const token = jwt.sign({ id: user.id, email: user.email },
14         ↪ process.env.JWT_SECRET, {
15             expiresIn: '7d',
16         });
17
18     await emailQueue.add('login', { type: 'login', to: user.email,
19         ↪ userName: user.firstName });
20
21     return { user, token };
22 } catch (error) {
23     throw error;
24 }
```

4.2.3 Asinkrona obdelava podatkov (Celery)

Za naloge, ki zahtevajo obsežnejšo obdelavo podatkov ali imajo daljši čas izvajanja, smo implementirali asinkrono obdelavo z uporabo sistema čakalnih vrst nalog. Asinkroni pristop omogoča, da strežnik takoj odgovori na zahtevek, medtem ko se zahtevnejše operacije izvajajo v ozadju kot ločeni procesi. Na ta način se zmanjšuje čakalni čas uporabnika in izboljša splošna odzivnost aplikacije [19]. Značilne naloge, ki zahtevajo asinkrono obdelavo, vključujejo zajemanje podatkov iz zunanjih virov, normalizacijo in transformacijo zajetih podatkov ter pošiljanje e-poštnih obvestil uporabnikom [19]. Za upravljanje čakalnih vrst nalog v okolju Node.js smo izbrali knjižnico BullMQ, ki zagotavlja zanesljivo upravljanje asinhronih nalog z vgrajeno podporo za ponovne poskuse (angl. *retry*), časovne omejitve (angl. *timeout*) in obravnavo napak [4]. Sistem temelji na kombinaciji podatkovnega skladišča Redis in delavskih procesov. Redis v tej arhitekturi služi kot posrednik sporočil (angl. *message broker*), ki hrani stanje nalog in omogoča koordinacijo med posameznimi delavskimi procesi [23].

Konfiguracija sistema zahteva tri ključne korake. Najprej se vzpostavi povezava s strežnikom Redis, ki deluje kot posrednik med čakalno vrsto in delavci. Nato se opredelijo posamezne vrste nalog (angl. *job types*) z ustreznimi funk-

cijami za njihovo izvajanje. Na koncu se registrirajo delavski procesi, ki poslušajo dodeljene vrste nalog, jih obdelujejo in ustrezno posodabljaajo stanje v skladišču Redis. S takšno zasnovo je zagotovljeno, da se dolgotrajne operacije ne izvajajo na glavni niti aplikacije, kar omogoča sočasno obdelavo in izboljšano razširljivost sistema.



Slika 4.3: Asinkrona obdelava podatkov z uporabo čakalnih vrst nalog in delavskih procesov [2].

4.3 Arhitektura uporabniškega vmesnika

Uporabniški vmesnik smo zasnovali iz več komponent, ki skupaj sestavljajo celoto. Pri tem smo upoštevali načela modularnosti, ki omogočajo ponovno uporabo komponent ter lažje vzdrževanje kode [33]. Vsako komponento smo

oblikovali tako, da opravlja natančno določeno nalogo in prikazuje omejen del vmesnika. S tem smo sledili načelu enotne odgovornosti, po katerem naj ima vsaka enota kode en sam razlog za spremembo [25]. Komponente smo name-noma ohranili majhne, saj majhen obseg naravno usmerja razvijalca k temu, da posamezna komponenta ostane osredotočena na eno samo funkcionalnost. Komponente si med seboj izmenjujejo podatke po hierarhiji nadrejenih in po-drejenih komponent. Nadrejena komponenta posreduje podatke podrejenim komponentam prek lastnosti (angl. *props*), podrejena komponenta pa nadre-jeni sporoča spremembe prek povratnih funkcij, ki jih nadrejena komponenta posreduje navzdol. Kot ogrodje za izgradnjo uporabniškega vmesnika smo izbrali React, ki je eno izmed najbolj razširjenih in uveljavljenih ogrodij za razvoj spletnih vmesnikov [26]. V naslednjih podpoglavjih podrobneje pred-stavimo posamezne dele te arhitekture.

4.3.1 Odjemalska shramba

Določene podatke si mora odjemalec zapomniti, kar pomeni, da jih mora shraniti na svoji strani. Prednost odjemalske shrambe je, da odjemalcu ni treba nenehno pošiljati novih zahtevkov na strežnik, kar občutno zmanjša njegovo obremenitev. Značilen primer uporabe je shranjevanje žetona JWT, ki ga uporabnik prejme ob prijavi in ga nato pri vsakem zahtevku pošilja v glavi [21]. Na ta način imamo podatke o prijavljenem uporabniku ves čas na voljo, dokler se ne odjavi ali dokler žeton JWT ne poteče.

Odjemalska shramba je uporabna tudi pri večkoračnih postopkih, saj si lahko zapomnimo, do katerega koraka je uporabnik prišel. Če uporabnik namerno ali nenamerno zapre stran, ostane njegovo napredovanje ohranjeno in lahko z delom nadaljuje. Pri shranjevanju podatkov na odjemalcu moramo biti pozorni, da hranimo le neobčutljive podatke ter da stanje pravilno posoda-bljamo ob vsaki spremembi. Spletni brskalniki za trajno hrambo podatkov na strani odjemalca ponujajo več mehanizmov, med drugim `localStorage` in `sessionStorage`[41]. V nasprotnem primeru se lahko stanje na odjemal-

čevi strani razlikuje od stanja na strežniku, kar vodi do težko odkrивljivih napak.

Za odjemalsko shrambo smo uporabili knjižnico Zustand, ki je preprosta, kompaktna in dobro vzdrževana. Zustand temelji na minimalističnem pristopu k upravljanju stanja in za osnovno delovanje ne zahteva ovojnih komponent (angl. *provider*), kar ga loči od nekaterih uveljavljenih rešitev, kot je Redux [**zustand-docs**]. Stanje je organizirano v obliki t.i. shramb (angl. *store*), do katerih dostopamo prek preprostih kavljev (angl. *hooks*), pri čemer se ponovno izrišejo le tiste komponente, ki dejansko uporabljajo spremenjeni del stanja. Zaradi majhne velikosti in neodvisnosti od ogrodja React je knjižnica primerna tudi za projekte, pri katerih želimo ohraniti nizko zahtevnost in dobro zmogljivost.

4.3.2 Pisanje zahtevkov API

Za pošiljanje zahtevkov smo uporabili knjižnico Axios, ki je ena izmed najbolj razširjenih knjižnic za pošiljanje zahtevkov HTTP v okolju JavaScript [3]. Nad njo smo izdelali ovojnico `api`, ki iz okoljskih spremenljivk (angl. *environment variables*) prebere naslov strežnika in ga uporabi kot osnovni naslov URL za vse zahteve. Tak pristop nam omogoča, da osnovni naslov in skupne nastavitve določimo na enem mestu, s čimer se izognemo ponavljanju in poenostavimo morebitne kasnejše spremembe.

Izsek kode 3: Ovojnica Axios za ustvarjanje odjemalca API.

```
1 import axios from "axios";
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_BASE_URL,
5   withCredentials: true,
6 });
7
8 export default api;
```

Ovojnico smo nato uporabili pri vsaki definiciji funkcije za pošiljanje zahtevka. Funkcija vrne polje `res.data`, ki vsebuje dejanski odgovor strežnika. Axios namreč vrne objekt s številnimi dodatnimi podatki, ki niso vedno potrebni, zato zaradi preglednosti kode vrnemo le vsebino odgovora[3]. Po potrebi lahko dostopamo tudi do statusne kode strežnika, glave odgovora in konfiguracije zahtevka.

Izsek kode 4: Primer definicije funkcije za pridobivanje znamk avtomobilov.

```
1 export const fetchCarBrands = async () => {  
2   const res = await api.get<{ brandNames: string[] }>("/api/cars/brands");  
3   return res.data;  
4 };
```

4.3.3 Vite kot razvojno in gradbeno orodje

Vsako izvorno kodo je treba pred prikazom v spletnem brskalniku ustrezno pripraviti oziroma zgraditi. To nalogo v našem projektu opravlja orodje Vite, ki omogoča hitro postavitev razvojnega okolja in gradnjo končne različice aplikacije [40]. Za razliko od starejših orodij za združevanje kode (angl. *bundler*), kot je Webpack, ki mora ob vsaki spremembi znova zgraditi večji del kode, Vite tega ne počne in je zato bistveno hitrejši.

Delovanje orodja Vite temelji na dveh ključnih načelih. Prvo je izraba native podpore modulom ECMAScript (angl. *ES modules*) v sodobnih brskalnikih. Ker sodobni brskalniki znajo neposredno razumeti module, uvožene v kodi, Vite med razvojem kode ne združuje, temveč posamezne module dostavi brskalniku po potrebi [29]. Drugo načelo je uporaba orodja esbuild za predhodno gradnjo odvisnosti JavaScript. Orodje esbuild je napisano v jeziku Go in je od 10 do 100-krat hitrejše od orodij za združevanje, napisanih v jeziku JavaScript [36].

Pomembna prednost orodja Vite je zmožnost sprotnega osveževanja modulov (angl. *Hot Module Replacement*), pri katerem se osveži le tisti del kode,

ki se je spremenil, brez izgube trenutnega stanja in podatkov aplikacije. Ta zmožnost je pri razvoju izjemno pomembna, saj občutno pospeši povratno zanko med spremembo in prikazom ter tako poveča produktivnost. Vite v naši arhitekturi torej opravlja vlogo razvojnega in gradbenega sloja.

4.4 Evalvacijski sistem

4.5 Infrastruktura in avtomatizacija

4.5.1 Več tabel ena ob drugi

Logika postavljanja dveh tabel ene ob drugo je popolnoma enaka kot pri slikah, le da znotraj okolja `table` uporabimo okolje `subtable`. Edino pomembno pravilo, na katerega ne smete pozabiti, je: **podnaslovi tabel vedno sodijo nad tabelo**, medtem ko podnaslovi slik sodijo pod njo.

Če ugotovite, da je vsebina tabele preširoka in povzroča prekrivanje, lahko celotno okolje `tabular` ovijete v ukaz `\resizebox{\linewidth}{!}{...}`, ki tabelo proporcionalno pomanjša, da se natančno prilega širini podtabele. Tabela 4.1 prikazuje ustrezen zapis na primeru vrednotenja modelov računalniškega vida.

Tabela 4.1: Primerjava uspešnosti klasifikacije slik z različnimi arhitekturami globokih nevronske mreže na dveh standardnih podatkovnih zbirkah. Pri metriki *točnost* (angl. *accuracy*) so boljše višje vrednosti (označeno s puščico ↑), pri metriki *napaka* (angl. *error rate*) pa so boljše nižje vrednosti (označeno s puščico ↓).

(a) Rezultati na zbirki CIFAR-10.

(b) Rezultati na zbirki ImageNet.

arhitektura	točnost (↑)	napaka (↓)	arhitektura	točnost (↑)	napaka (↓)
ResNet-18	93,5 %	6,5 %	ResNet-18	69,7 %	30,3 %
ResNet-50	95,2 %	4,8 %	ResNet-50	76,1 %	23,9 %
ViT-Base	97,8 %	2,2 %	ViT-Base	84,2 %	15,8 %

Poglavje 5

Vključevanje programske kode in algoritmov

Svojo programsko kodo v poljubnem programskem jeziku najlažje vključimo s paketom `minted`. Okoli bloka `\minted{ }` sicer lahko podamo klasični `\listing{ }` in skupaj z dodanim `\caption` dobimo vse potrebno za prikaz kode, vendar pa se takšen blok ne razteza pravilno skozi več strani. Ena od možnosti za rešitev je uporaba bloka `\center{ }`, za napis pa `\captionof{ }`, kot je prikazano pri obeh primerih izsekov kode 5 in 6.

Izsek kode 5: Kratek primer kode v programskem jeziku Python.

```
1 def python_veverica():  
2     # Oh, kako je to lepo!  
3     x = -1  
4     print("Res izgleda čudovito.")
```

Kar je nadvse čudovito pri paketu `minted` je to, da lahko podamo dele kode znotraj vrstice, kot je npr. tole: `python_veverica()`.

Izsek kode 6: Malo daljši primer programske kode v jeziku Javascript, ki se razteza skozi dve strani.

```
1 async function pridobiUporabnika(id) {
2   const url = `https://api.primer.fri/users/${id}`;
3
4   try {
5     const odgovor = await fetch(url);
6
7     // Preverimo, če je statusni kod v območju 200-299
8     if (!odgovor.ok) {
9       console.error(`Napaka pri povezavi: ${odgovor.status}`);
10      throw new Error(`Uporabnika z ID ${id} ni bilo mogoče
11        ↳ najti.`);
12    }
13
14    const podatki = await odgovor.json();
15
16    // Logiranje uspešnega prejema podatkov
17    console.log("Podatki so bili uspešno prejeti.");
18    console.log(`Ime uporabnika: ${podatki.name}`);
19
20    return podatki;
21  } catch (napaka) {
22    // Tukaj ulovimo tako mrežne napake kot napake pri branju JSON-a
23    console.warn("Prišlo je do nepričakovane težave.");
24    return { error: napaka.message };
25  }
26 }
```

Vodoravni črti pri začetku in koncu izseka, v kombinaciji s sivim ozadjem, vizualno sporočata morebitno raztezanje skozi več strani. To je razvidno pri izseku kode 6.

5.1 Vključevanje algoritmov (psevdokode)

Medtem ko paket `minted` uporabljamo za prikaz dejanske implementacije v specifičnem programskem jeziku, v diplomskih nalogah pogosto opisujemo samo logiko reševanja problema na višjem nivoju. Za zapis algoritmov in obliki psevdokode je standardna izbira kombinacija paketov `algorithm` in `algpseudocode` (slednji je del paketa `algorithmicx`).

Za razliko od izsekov izvirne kode, algoritme običajno obravnavamo kot standardne plavajoče objekte (tako kot slike in tabele). To pomeni, da $\text{\LaTeX}$ sam poišče optimalno mesto za njihov prikaz na strani. V besedilu se nanje sklicujemo z nedeljivim presledkom in ukazom `\ref`, na primer: Algoritem 1 prikazuje klasičen postopek za iskanje največjega skupnega delitelja.

Naslov (`\caption`) pri algoritmih običajno postavljamo na vrh, tik pod začetek okolja `algorithm`. Številka [1] ob ukazu `\begin{algorithmic}` pa $\text{\LaTeX}$ u sporoči, naj oštevilči vsako vrstico algoritma, kar nam olajša sklicovanje na posamezne korake v besedilu.

Algorithm 1 Evklidov algoritem za iskanje največjega skupnega delitelja

Require: $a \geq 0 \vee b \geq 0$ **Ensure:** Največji skupni delitelj a in b

```
1:
2: procedure EVKLID( $a, b$ )
3:   while  $b \neq 0$  do
4:      $t \leftarrow b$ 
5:      $b \leftarrow a \bmod b$ 
6:      $a \leftarrow t$ 
7:   end while
8:   return  $a$ 
9: end procedure
```

Poglavje 6

Kaj pa literatura?

Kot smo omenili že v uvodu, je pravi način za citiranje literature uporaba `BIBLATEX`a. `BIBLATEX` zagotovi, da pri določeni vrsti literature ne izpustimo nobene obvezne informacije in da vse informacije dosledno navajamo na enak način in po istem vrstnem redu. `BIBLATEX` je nadgradnja starejšega sistema `BIBTEX`. Novejši sistem je bolj prilagojen slovenščini in navajanju spletnih virov. Sicer pa so starejše datoteke `.bib` kompatibilne z `BIBLATEX`om.

Osnovna ideja `BIBLATEX`a je, da vse informacije o literaturi zapisujemo v posebno datoteko, v našem primeru je to `literatura.bib`. Vsakemu viru v tej datoteki določimo simbolično ime. V našem primeru je v tej datoteki nekaj najbolj značilnih zvrsti literature, kot so knjige članki v revijah in zbornikih konferenc poglavja v knjigah spletni viri tehnično poročilo diplome itd. Diploma iz leta 1990 je bila prva diploma na tedanji Fakulteti za elektrotehniko in računalništvo, ki je bila oblikovana z `LATEX`om. Reference, ki so na spletnih straneh arhivirane v elektronski obliki, imajo običajno številko DOI (<http://dx.doi.org>), ki jo zato tudi vključimo v izpis literature in tako bralcu elektronske verzije naše publikacije ponudimo neposredno povezavo do elektronske kopije te reference.

Po vsaki spremembi pri sklicu na literaturo moramo najprej prevesti izvirno besedilo s prevajalnikom `LATEX`, nato s prevajalnikom `BIBLATEX`, ki ustvari datoteko `vzorec_dip_Seminar.bbl`, in nato še dvakrat s prevajalni-

kom $\text{\LaTeX}$. V okolju Overleaf je to večkratno prevajanje z različnimi prevajalniki uporabniku skrito. Zato tudi začetnim uporabnikom $\text{\LaTeX}$ a svetujemo uporabo Overleafa.

Kako se spisek literature nato izpiše (ali so posamezni viri razvrščeni po vrstnem redu sklicevanja, ali po abecedi priimkov prvih avtorjev, ali se imena avtorjev pišejo pred priimki itd.) je odvisno od parametrov paketa $\text{BIB}\text{\LaTeX}$. V diplomu bomo uporabili parametre `style=numeric`, kar pomeni, da bodo sklici na literaturo v besedilu označeni z zaporednimi številkami, za vrstni red izpisa referenc pa `sorting=nty`, kar pomeni, da bodo reference urejene po priimkih prvih avtorjev, nato po naslovu reference in nazadnje po letu izdaje. Zato je potrebno pri določenih zvrsteh literature, ki nima avtorjev, dodati parameter `key`, ki določi vrstni red vira po abecedi.

Ko začenjamo uporabljati $\text{BIB}\text{\LaTeX}$ je lažje, če za urejanje datoteke `.bib` uporabljamo kar isti urejevalnik kot za urejanje datotek `.tex`, čeprav obstajajo tudi posebni urejevalniki oziroma programi za delo z datotekami `.bib`.

Le če se na določen vir v besedilu tudi sklicujemo, se bo ta vir pojavil tudi v spisku literature. Tako je avtomatično zagotovljeno, da se na vsak vir v seznamu literature tudi sklicujemo v besedilu diplome. V datoteki `.bib` imamo sicer lahko veliko več virov za literaturo, kot jih bomo uporabili v diplomu.

Poglavje 7

Sklepne ugotovitve

Uporaba L^AT_EXa in BIBL^AT_EXa je pri diplomski nalogi **obvezna**!

Preden diplomo oddate mentorjem, še enkrat preverite:

- pravilno rabo elementov (tako L^AT_EXgradnikov kot tudi ločil kratic, pomišljajev itd.),
- slovnico (lahko si pomagate s katerim od orodij),
- da ni nobeno besedilo prilepljeno direktno iz velikega jezikovnega modela (čisto vse morate prebrati in preveriti, če že ne popraviti),
- da je dovolj slikovnih elementov (preveč suhoparnega besedila ni za nikogar koristno),
- da je vsaka slika in tabela ne samo podnaslovljena, ampak tudi referencirana v besedilu.

Preden diplomo oddate na sistemu STUDIS, še enkrat preverite:

- če je izpisana pravilna smer študija, vaše ime, ime in naziv mentorja in somentorja,
- če so vse besede pravilno deljene in da ne segajo preko desnega roba (poravnano po vrsticah lahko kontrolirate tako, da izvirno datoteko enkrat testno prevedete z opcijo **draft**, kar vam pokaže predolge vrstice),

- če ste vključili pravilne verzije slik, predvsem pri grafih in diagramih vektorske (.pdf ali .eps) in ne rasterskih (npr. .jpg ali .png),
- če ste naslovili vse napake in opozorila, ki se lahko pojavijo pri tvorjenju dokumenta (bodite pozorni predvsem na manjkajoče citate, ki jih lahko poiščete tudi z iskanjem vprašajev v končnem dokumentu).

Literatura

Članki v revijah

- [7] Iwan Darmawan in sod. „Evaluating Web Scraping Performance Using XPath, CSS Selector, Regular Expression, and HTML DOM With Multiprocessing Technical Applications“. V: *JOIV: International Journal on Informatics Visualization* (2022). DOI: 10.30630/joiv.6.4.1525.
- [9] Edsger W. Dijkstra. „On the role of scientific thought“. V: *ACM Symposium on Principles of Programming Languages* (1974), str. 1–6. DOI: 10.1145/942572.807378.
- [16] Hector Garcia-Molina, Jeffrey D. Ullman in Jennifer Widom. „Database Systems: The Complete Book“. V: (2008), str. 587–625.
- [19] John L. Hennessy in David A. Patterson. „Computer Architecture: A Quantitative Approach“. V: (2019), str. 412–450.
- [20] Marijn Janssen, Yannis Charalabidis in Anneke Zuiderwijk. „Benefits, Adoption Barriers and Myths of Open Data and Open Government“. V: *Information Systems Management* 29.4 (2012), str. 258–268. DOI: 10.1080/10580530.2012.716740.
- [24] Alberto H. F. Laender in sod. „A Brief Survey of Web Data Extraction Tools“. V: *ACM SIGMOD Record* 31.2 (2002), str. 84–93. DOI: 10.1145/565117.565137.

- [31] Christopher Olston in Marc Najork. „Web Crawling“. V: *Foundations and Trends in Information Retrieval* 4.3 (2010), str. 175–246. DOI: 10.1561/15000000017.

Članki v zbornikih konferenc

- [17] Enis Gegic in sod. „Car Price Prediction using Machine Learning Techniques“. V: *TEM Journal*. Zv. 8. 1. 2019, str. 113–118. DOI: 10.18421/TEM81-16.
- [34] Niels Provos in David Mazières. „A Future-Adaptable Password Scheme“. V: *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. 1999, str. 81–91.

Ostala literatura

- [1] V: *Encyclopedic Dictionary of Archaeology*. Springer International Publishing, 2021, str. 1150–1150. ISBN: 9783030582920. DOI: 10.1007/978-3-030-58292-0_180189. URL: http://dx.doi.org/10.1007/978-3-030-58292-0_180189.
- [2] 21je0710. *Task Queues and Background Jobs: A Backend Developer's Guide*. Dostopano: 11. 7. 2026. 2023. URL: <https://medium.com/@21je0710/task-queues-and-background-jobs-a-backend-developers-guide-470d22d52666>.
- [3] Axios Contributors. *Axios – Promise based HTTP client for the browser and node.js*. <https://axios-http.com>. Uradna dokumentacija knjižnice. 2024.
- [4] BullMQ Contributors. *BullMQ Documentation*. Dostopano: junij 2024. 2024. URL: <https://docs.bullmq.io/>.
- [5] Express.js Contributors. *Express.js Documentation*. Dostopano: junij 2024. 2024. URL: <https://expressjs.com/>.
- [6] Sequelize Contributors. *Sequelize Documentation*. Dostopano: junij 2024. 2024. URL: <https://sequelize.org/>.
- [8] Ali Darvish Darab. *RESTful API – CS421 Course Notes*. Dostopano: 28. 6. 2026. 2020. URL: https://darvishdarab.github.io/cs421_f20/docs/readings/restful/api/.
- [10] Nicola Dragoni in sod. „Microservices: Yesterday, Today, and Tomorrow“. V: *Present and Ulterior Software Engineering*. Springer International Publishing, 2017, str. 195–216. ISBN: 9783319674254. DOI: 10.1007/978-3-319-67425-4_12.
- [11] Ramez Elmasri in Shamkant B. Navathe. *Fundamentals of Database Systems*. 7. izd. Pearson, 2015. ISBN: 9780133970777.
- [12] Eurostat. *Eurostat – European Statistics*. Dostopano: junij 2025. 2024. URL: <https://ec.europa.eu/eurostat>.

- [13] F5, Inc. *Rate Limiting with NGINX and NGINX Plus*. Dostopano: 21. 6. 2026. 2024. URL: <https://www.nginx.com/blog/rate-limiting-nginx/>.
- [14] Roy Thomas Fielding. „Architectural Styles and the Design of Network-based Software Architectures“. Doktorska disertacija. University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [15] David Flanagan. *JavaScript: The Definitive Guide*. 7. izd. O'Reilly Media, 2020. ISBN: 9781491952023.
- [18] IETF OAuth Working Group. *JWT Best Current Practices*. Dostopano: junij 2024. 2023. URL: <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-jwt-bcp-07>.
- [21] Michael Jones, John Bradley in Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. Maj 2015. DOI: 10.17487/RFC7519. URL: <https://tools.ietf.org/html/rfc7519>.
- [22] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. 1. izd. O'Reilly Media, 2017. ISBN: 978-1449373320.
- [23] Redis Labs. *Redis Documentation*. Dostopano: junij 2024. 2024. URL: <https://redis.io/docs/>.
- [25] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 9780134494166.
- [26] Meta Platforms, Inc. *React – The library for web and native user interfaces*. <https://react.dev>. Uradna dokumentacija. 2024.
- [27] Meta Platforms, Inc. *React Documentation: Describing the UI*. Dostopano: 21. 6. 2026. 2024. URL: <https://react.dev/learn/describing-the-ui>.
- [28] Microsoft. *TypeScript: JavaScript With Syntax For Types*. Dostopano: junij 2025. 2024. URL: <https://www.typescriptlang.org/>.

- [29] Mozilla Developer Network. *JavaScript modules*. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>. Tehnična dokumentacija. 2024.
- [30] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015. ISBN: 9781491950357.
- [32] Orgesleka. *Used Cars Database – eBay Kleinanzeigen*. Dostopano: junij 2025. 2016. URL: <https://www.kaggle.com/datasets/orgesleka/used-cars-database>.
- [33] David L. Parnas. *On the Criteria To Be Used in Decomposing Systems into Modules*. Zv. 15. 12. Association for Computing Machinery, 1972, str. 1053–1058. DOI: 10.1145/361598.361623. URL: <https://doi.org/10.1145/361598.361623>.
- [35] Leonard Richardson in Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2007. ISBN: 9780596529260.
- [36] Evan Wallace Sanders. *esbuild – An extremely fast bundler for the web*. <https://esbuild.github.io>. Uradna dokumentacija orodja. 2024.
- [37] Statistični urad Republike Slovenije. *Statistični urad Republike Slovenije – SURS*. Dostopano: junij 2025. 2024. URL: <https://www.stat.si/StatWeb/>.
- [38] The PostgreSQL Global Development Group. *About PostgreSQL*. Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/about/>.
- [39] The PostgreSQL Global Development Group. *PostgreSQL Documentation: What Is PostgreSQL?* Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/docs/current/intro-what-is.html>.
- [40] Vite Contributors. *Vite – Next Generation Frontend Tooling*. <https://vite.dev>. Uradna dokumentacija orodja. 2024.

-
- [41] WHATWG. *HTML Living Standard – Web Storage*. <https://html.spec.whatwg.org/multipage/webstorage.html>. Uradna specifikacija. 2024.

Dodatek

Koda: Definicija modela User

Izsek kode 7: Definicija modela `User` z ORM knjižnico Sequelize.

```
1 import { DataTypes } from "sequelize";
2 import sequelize from "../config/db.js";
3
4 const User = sequelize.define(
5   "User",
6   {
7     id: {
8       type: DataTypes.INTEGER,
9       autoIncrement: true,
10      primaryKey: true,
11    },
12    firstName: {
13      type: DataTypes.STRING,
14      allowNull: false,
15    },
16    lastName: {
17      type: DataTypes.STRING,
18    },
19    email: {
20      type: DataTypes.STRING,
21      allowNull: false,
22      unique: true,
23      validate: {
24        isEmail: true,
25      },
26    },
27    telephone: {
28      type: DataTypes.STRING,
```

```
29     allowNull: false,
30   },
31   password: {
32     type: DataTypes.STRING,
33     allowNull: false,
34   },
35   soldCars: {
36     type: DataTypes.INTEGER,
37     defaultValue: 0,
38   },
39 },
40 { timestamps: true },
41 );
42
43 export default User;
```
